

Data Vortex A'26

Round 3 — Data Collection Scripts

WhatsApp Privacy Policy 2021: Backlash, Sentiment & Migration Analysis

Team: Event Horizon

Total Records Collected: 14,891 across 4 sources

Date Coverage: Nov 2020 – Jul 2021

Table of Contents

1. GDELT Global News
2. Hacker News (Algolia API)
3. Kaggle Play Store Reviews
4. Google Play Store Reviews
5. Reddit (PRAW)

1. GDELT Global News

◆ *Live API • No Auth Required • 3,929 articles collected*

GDELT (Global Database of Events, Language, and Tone) is one of the world's largest open databases of human society, updated every 15 minutes. It indexes news articles from thousands of global sources across hundreds of languages.

We used the GDELT DOC 2.0 API to pull English-language news articles mentioning WhatsApp privacy keywords across our full date window (Nov 2020 – Jul 2021). The collector queries four themed search terms in 28-day batches, enforcing GDELT's mandatory 5-second politeness delay between every request. Results are saved incrementally and de-duplicated by URL to prevent double-counting across overlapping query windows.

■ Collector Script: `gdelt_collector.py`

```
"""
gdelt_collector.py - GDELT 2.0 DOC API Collector (Optimized 5.5s Politeness + Monthly
Ingestion)
Data Vortex A'26 | Team: Event Horizon

Fetches global news articles mentioning WhatsApp privacy keywords
from GDELT DOC 2.0 API with strict adherence to the 5-second rate limit.
Saves incrementally to round3/data/raw/gdelt_live_stream.csv.
"""

import sys, os
sys.path.insert(0, os.path.join(os.path.dirname(__file__), ".."))

import time
import requests
import pandas as pd
from datetime import datetime, timedelta, timezone
from config import (
    DATE_START, DATE_END,
    DATA_RAW_DIR, COLL_LOG_PATH
)

GDELT_DOC_URL = "https://api.gdeltproject.org/api/v2/doc/doc"

GDELT_THEMES = [
    "whatsapp privacy policy",
    "whatsapp new terms",
    "whatsapp privacy",
    "telegram whatsapp privacy"
]

OUT_FILE = os.path.join(DATA_RAW_DIR, "gdelt_live_stream.csv")

def log_collection(source, query, count, status, run_ts=None):
    run_ts = run_ts or datetime.now(timezone.utc).isoformat()
    row = pd.DataFrame([
        {
            "timestamp": run_ts,
            "source": source,
            "query": query,
            "count": count,
            "status": status,
        }
    ])
    if os.path.exists(COLL_LOG_PATH):
        row.to_csv(COLL_LOG_PATH, mode="a", header=False, index=False)
    else:
        os.makedirs(os.path.dirname(COLL_LOG_PATH), exist_ok=True)
        row.to_csv(COLL_LOG_PATH, index=False)

def query_gdelt_window(query: str, start_dt: datetime, end_dt: datetime, maxrecords: int = 250)
    -> list:
    params = {
```

```

        "query": query,
        "mode": "artlist",
        "maxrecords": maxrecords,
        "startdatetime": start_dt.strftime("%Y%m%d%H%M%S"),
        "enddatetime": end_dt.strftime("%Y%m%d%H%M%S"),
        "format": "json",
        "sort": "DateDesc",
        "sourcelang": "english",
    }
}

max_retries = 3
for attempt in range(max_retries):
    try:
        resp = requests.get(GDELT_DOC_URL, params=params, timeout=25)
        if resp.status_code == 429:
            wait_time = 7.0 * (attempt + 1)
            print(f" [GDELT] Rate limit hit (429). Waiting {wait_time}s...")
            time.sleep(wait_time)
            continue

        resp.raise_for_status()
        data = resp.json()
        return data.get("articles", [])
    except Exception as e:
        if attempt == max_retries - 1:
            print(f" [GDELT] Failed query '{query}': {e}")
            return []
        time.sleep(6.0)

return []

def append_to_csv(records):
    if not records:
        return
    df_new = pd.DataFrame(records)
    if os.path.exists(OUT_FILE):
        df_existing = pd.read_csv(OUT_FILE, low_memory=False)
        df_combined = pd.concat([df_existing, df_new], ignore_index=True)
        df_combined.drop_duplicates(subset=["post_id"], inplace=True)
        df_combined.to_csv(OUT_FILE, index=False)
    else:
        df_new.drop_duplicates(subset=["post_id"], inplace=True)
        df_new.to_csv(OUT_FILE, index=False)

def collect_gdelt(start: str = DATE_START, end: str = DATE_END, batch_days: int = 28) ->
pd.DataFrame:
    os.makedirs(DATA_RAW_DIR, exist_ok=True)
    run_ts = datetime.now(timezone.utc).isoformat()

    start_dt = datetime.strptime(start, "%Y-%m-%d").replace(tzinfo=timezone.utc)
    end_dt = datetime.strptime(end, "%Y-%m-%d").replace(tzinfo=timezone.utc)

    total_added = 0
    current = start_dt

    while current < end_dt:
        window_end = min(current + timedelta(days=batch_days), end_dt)
        batch_records = []

        for query in GDELT_THEMES:
            print(f" [GDELT] Querying {current.date()} to {window_end.date()} | Theme:
                '{query}'...")
            articles = query_gdelt_window(query, current, window_end)
            print(f" -> Received {len(articles)} articles.")
            for a in articles:
                batch_records.append({
                    "post_id": a.get("url", ""),
                    "source": "gdelt",
                    "source_type": "bonus_live_scraped",
                    "source_name": "gdelt",
                    "text": a.get("title", ""),
                    "url": a.get("url", ""),
                    "timestamp": a.get("seendate", ""),
                    "engagement_metric": 0.0,

```

```

        "country":          a.get("sourcecountry", ""),
        "lang":             a.get("language", "English"),
        "query_used":       query,
    })
    time.sleep(5.5) # Crucial: GDELT requires >5s between requests

    if batch_records:
        append_to_csv(batch_records)
        total_added += len(batch_records)
        if os.path.exists(OUT_FILE):
            cur_len = len(pd.read_csv(OUT_FILE, low_memory=False))
            print(f" [GDELT] Saved! File now has {cur_len:,} total deduplicated records ->
                  {OUT_FILE}")

        current = window_end

    print(f" [GDELT] Finished all date ranges!")
    log_collection("gdelt", "multi-query batch", total_added, "OK", run_ts)

    if os.path.exists(OUT_FILE):
        return pd.read_csv(OUT_FILE)
    return pd.DataFrame()

if __name__ == "__main__":
    print("=== GDELT Collector (Strict 5.5s Rate Limit + Incremental Saving) ===")
    collect_gdelt()

```

2. Hacker News (Algolia API)

◆ *Live API • No Auth Required • 6,149 posts collected*

Hacker News is the premier tech-community discussion platform, run by Y Combinator. Its audience skews toward developers, researchers, and privacy-conscious power users — making it an ideal signal source for tech-savvy sentiment around WhatsApp's policy changes.

We used the free Algolia HN Search API (no authentication required) to fetch all stories and comments matching our four query terms within the Jan–Jul 2021 window. HTML entities returned by the API are cleaned inline before saving. The collector is the largest single source in our dataset with 6,149 records.

■ Collector Script: `hn_collector.py`

```
"""
hn_collector.py - Hacker News Algolia API Collector
Data Vortex A'26 | Team: Event Horizon

Fetches comments and stories mentioning WhatsApp privacy policy updates.
Uses the free Algolia HN Search API (no auth required).
Excellent source for tech-community sentiment on privacy matters.
"""

import sys, os
sys.path.insert(0, os.path.join(os.path.dirname(__file__), ".."))

import time
import requests
import pandas as pd
from datetime import datetime, timezone
from config import (
    DATE_START, DATE_END,
    DATA_RAW_DIR, COLL_LOG_PATH
)

HN_API_URL = "https://hn.algolia.com/api/v1/search_by_date"
HN_QUERIES = ["whatsapp privacy", "whatsapp policy", "whatsapp terms", "signal telegram"]

def log_collection(source, query, count, status, run_ts=None):
    run_ts = run_ts or datetime.now(timezone.utc).isoformat()
    row = pd.DataFrame([
        {
            "timestamp": run_ts,
            "source": source,
            "query": query,
            "count": count,
            "status": status,
        }
    ])
    if os.path.exists(COLL_LOG_PATH):
        row.to_csv(COLL_LOG_PATH, mode="a", header=False, index=False)
    else:
        os.makedirs(os.path.dirname(COLL_LOG_PATH), exist_ok=True)
        row.to_csv(COLL_LOG_PATH, index=False)

def collect_hn(start: str = DATE_START, end: str = DATE_END) -> pd.DataFrame:
    os.makedirs(DATA_RAW_DIR, exist_ok=True)
    all_records = []
    run_ts = datetime.now(timezone.utc).isoformat()

    start_ts = int(datetime.strptime(start,
        "%Y-%m-%d").replace(tzinfo=timezone.utc).timestamp())
    end_ts = int(datetime.strptime(end, "%Y-%m-%d").replace(tzinfo=timezone.utc).timestamp())

    for query in HN_QUERIES:
        print(f" [HN] Querying: '{query}'")
        params = {
            "query": query,
            "numericFilters": f"created_at_i>{start_ts},created_at_i<{end_ts}",
            "hitsPerPage": 1000, # Max allowed by Algolia
        }
```

```

}

try:
    resp = requests.get(HN_API_URL, params=params, timeout=30)
    resp.raise_for_status()
    data = resp.json()
    hits = data.get("hits", [])

    for h in hits:
        # Text can be in 'story_text', 'comment_text', or just 'title'
        text = h.get("comment_text") or h.get("story_text") or h.get("title") or ""

        # Basic cleaning of HTML tags commonly returned by HN Algolia API
        text = text.replace("<p>", " ").replace("</p>", " ").replace("&#x2F;",
            "/").replace("&quot;", "'")

        if not text.strip():
            continue

        all_records.append({
            "post_id":          str(h.get("objectID", "")),
            "source":           "hacker_news",
            "source_type":      "bonus_live_scraped",
            "source_name":      "hacker_news",
            "text":             text,
            "url":              f"https://news.ycombinator.com/item?id={h.get('objectID')}",
            "timestamp":        h.get("created_at", ""),
            "engagement_metric": float(h.get("points") or 0), # Points (upvotes)
            "country":          "Global", # HN is global tech community
            "lang":             "en",
            "query_used":       query,
        })
        time.sleep(1) # Be polite
except Exception as e:
    print(f" [HN] Error fetching '{query}': {e}")

df = pd.DataFrame(all_records)
if not df.empty:
    df.drop_duplicates(subset=["post_id"], inplace=True)
    ts_label = datetime.now(timezone.utc).strftime("%Y%m%dT%H%M%S")
    out_path = os.path.join(DATA_RAW_DIR, f"hn_{ts_label}.csv")
    df.to_csv(out_path, index=False)
    print(f" [HN] Saved {len(df):,} records -> {out_path}")
    log_collection("hacker_news", "multi-query batch", len(df), "OK", run_ts)
else:
    print(" [HN] No records returned.")
    log_collection("hacker_news", "multi-query batch", 0, "EMPTY", run_ts)

return df

if __name__ == "__main__":
    print("=== Hacker News Collector ===")
    df = collect_hn()
    print(f"Total rows collected: {len(df):,}")

```

3. Kaggle Play Store Reviews

◆ *External Dataset • Multilingual • ~4,800 reviews processed*

Two large Kaggle datasets were ingested as primary data sources: a WhatsApp Play Store review dump (760 MB, millions of rows) and a Telegram Play Store review dump containing Indonesian-language reviews from the privacy backlash period.

The collector applies a five-stage pipeline: (1) strict date range filter to Jan–Jul 2021, (2) keyword noise filter (English + Indonesian terms: privacy, privasi, kebijakan, telegram, signal, etc.), (3) random sampling capped at 3,000 rows per dataset to respect translation API limits, (4) batch translation of Indonesian text to English using deep_translator (Google Translate), and (5) schema harmonisation to our pipeline's standard column format.

■ Collector Script: `kaggle_collector.py`

```
"""
kaggle_collector.py - Ingest, Filter, Translate, and Harmonize Kaggle Play Store Datasets
Data Vortex A'26 | Team: Event Horizon

Handles large external Kaggle CSVs (e.g. WhatsApp / Telegram Play Store Reviews).
Applies:
1. Strict Date Range Filter: 2021-01-01 to 2021-07-31
2. Noise Reduction Keyword Filter: Privacy, Policy, Security, Terms, Data Sharing, Migration
   terms (ID & EN)
3. Sampling: Caps max rows per dataset to prevent API translation rate limits
4. Batch Translation: Translates Indonesian (id) text to English (en) using deep_translator
5. Harmonization: Standardizes columns to pipeline schema
"""

import sys, os
sys.path.insert(0, os.path.join(os.path.dirname(__file__), ".."))

import time
import pandas as pd
from datetime import datetime, timezone
from config import DATA_RAW_DIR, COLL_LOG_PATH

try:
    from deep_translator import GoogleTranslator
    HAS_TRANSLATOR = True
except ImportError:
    HAS_TRANSLATOR = False

KAGGLE_DIR = os.path.join(os.path.dirname(__file__), "..", "..", "data", "kaggle")

# Keywords for noise filtering (English + Indonesian)
RELEVANT_KEYWORDS = [
    "privasi", "privacy", "kebijakan", "policy", "syarat", "terms",
    "ketentuan", "tos", "data", "facebook", "fb", "telegram", "signal",
    "pindah", "switch", "migrate", "update", "keamanan", "security",
    "aturan", "ganti", "hapus", "delete", "akun", "account", "sharing",
    "share", "ijin", "izin", "personal", "pribadi", "bocor", "leak"
]

def log_collection(source, query, count, status, run_ts=None):
    run_ts = run_ts or datetime.now(timezone.utc).isoformat()
    row = pd.DataFrame([
        {
            "timestamp": run_ts,
            "source": source,
            "query": query,
            "count": count,
            "status": status,
        }
    ])
    if os.path.exists(COLL_LOG_PATH):
        row.to_csv(COLL_LOG_PATH, mode="a", header=False, index=False)
    else:
```

```

        os.makedirs(os.path.dirname(COLL_LOG_PATH), exist_ok=True)
        row.to_csv(COLL_LOG_PATH, index=False)

def translate_batch(texts, max_retries=3):
    """Translate a list of Indonesian texts to English using deep_translator."""
    if not HAS_TRANSLATOR:
        print(" [Kaggle] Warning: deep_translator not installed. Keeping original text.")
        return texts

    translator = GoogleTranslator(source='auto', target='en')
    translated = []
    print(f" [Kaggle] Translating {len(texts):,} records to English...")

    for i, t in enumerate(texts):
        if not isinstance(t, str) or not t.strip():
            translated.append("")
            continue

        # Avoid unnecessary translation for short non-text
        if len(t) < 3:
            translated.append(t)
            continue

        res = t
        for attempt in range(max_retries):
            try:
                res = translator.translate(t)
                break
            except Exception as e:
                if attempt == max_retries - 1:
                    res = t
                time.sleep(0.5)

        translated.append(res)
        if (i + 1) % 100 == 0 or (i + 1) == len(texts):
            print(f"    Translated {i + 1}/{len(texts)} rows...")
            time.sleep(0.2) # Polite API delay

    return translated

def process_kaggle_csv(file_path: str, date_start="2021-01-01", date_end="2021-07-31",
max_sample=3000):
    """Filter, sample, translate, and standardize a Kaggle review CSV."""
    if not os.path.exists(file_path):
        print(f" [Kaggle] File not found: {file_path}")
        return pd.DataFrame()

    print(f"\n [Kaggle] Reading dataset: {os.path.basename(file_path)}")
    df = pd.read_csv(file_path, low_memory=False)
    print(f" [Kaggle] Raw row count: {len(df):,}")

    # Detect text column
    text_col = None
    for c in ["content", "review", "text", "body", "comment", "Review"]:
        if c in df.columns:
            text_col = c
            break
    if not text_col:
        print(" [Kaggle] Could not find text column in CSV!")
        return pd.DataFrame()

    # Detect date column
    date_col = None
    for c in ["at", "date", "created_at", "timestamp", "Time", "Date"]:
        if c in df.columns:
            date_col = c
            break
    if not date_col:
        print(" [Kaggle] Could not find date column in CSV!")
        return pd.DataFrame()

    # 1. Date Range Filtering
    df[date_col] = pd.to_datetime(df[date_col], errors="coerce", utc=True)
    df = df.dropna(subset=[date_col])

```

```

start_dt = pd.to_datetime(date_start, utc=True)
end_dt = pd.to_datetime(date_end, utc=True)

df_filtered = df[(df[date_col] >= start_dt) & (df[date_col] <= end_dt)].copy()
print(f" [Kaggle] Rows within 2021 date range ({date_start} to {date_end}):
      {len(df_filtered):,}")

if df_filtered.empty:
    print(" [Kaggle] No rows match the specified date range.")
    return pd.DataFrame()

# 2. Keyword / Noise Filtering
pattern = "|".join(RELEVANT_KEYWORDS)
df_filtered = df_filtered[df_filtered[text_col].astype(str).str.contains(pattern,
    case=False, na=False)]
print(f" [Kaggle] Rows matching privacy/policy/migration keywords: {len(df_filtered):,}")

if df_filtered.empty:
    print(" [Kaggle] No rows match the privacy keyword filters.")
    return pd.DataFrame()

# 3. Sampling to prevent API translation limits
if len(df_filtered) > max_sample:
    df_filtered = df_filtered.sample(n=max_sample, random_state=42)
    print(f" [Kaggle] Sampled down to {len(df_filtered):,} rows for processing &
          translation.")

# 4. Translation
translated_texts = translate_batch(df_filtered[text_col].tolist())
df_filtered["translated_text"] = translated_texts

# 5. Schema Harmonization
app_source = "kaggle_whatsapp" if "whatsapp" in file_path.lower() else "kaggle_telegram"

score_col = None
for c in ["score", "rating", "star", "stars", "Rating"]:
    if c in df_filtered.columns:
        score_col = c
        break

records = []
for idx, row in df_filtered.iterrows():
    records.append({
        "post_id":      f"kaggle_{idx}",
        "source":       app_source,
        "text":         row["translated_text"],
        "url":          "https://kaggle.com/datasets",
        "timestamp":    row[date_col].isoformat(),
        "engagement_metric": float(row[score_col]) if score_col else 1.0,
        "country":      "ID", # Indonesian dataset
        "lang":         "id-en",
        "query_used":   "kaggle_play_store_reviews",
    })

out_df = pd.DataFrame(records)

# Save output
os.makedirs(DATA_RAW_DIR, exist_ok=True)
ts_label = datetime.now(timezone.utc).strftime("%Y%m%dT%H%M%S")
out_path = os.path.join(DATA_RAW_DIR, f"{app_source}_{ts_label}.csv")
out_df.to_csv(out_path, index=False)
print(f" [Kaggle] Saved {len(out_df):,} clean translated records -> {out_path}")

log_collection(app_source, "kaggle_ingest", len(out_df), "OK")
return out_df

def process_all_kaggle():
    """Scans the round3/data/kaggle directory for any CSV files and processes them."""
    if not os.path.exists(KAGGLE_DIR):
        os.makedirs(KAGGLE_DIR, exist_ok=True)
        print(f" [Kaggle] Created folder: {KAGGLE_DIR}. Place Kaggle CSV files here!")
        return

    csv_files = [os.path.join(KAGGLE_DIR, f) for f in os.listdir(KAGGLE_DIR) if

```

```
        f.endswith(".csv")]  
if not csv_files:  
    print(f" [Kaggle] No CSV files found in {KAGGLE_DIR}.")  
    return  
  
for f in csv_files:  
    process_kaggle_csv(f)  
  
if __name__ == "__main__":  
    print("=== Kaggle Dataset Ingestion & Translation Collector ===")  
    process_all_kaggle()
```

4. Google Play Store Reviews

◆ *Live Scrape • No Auth Required • Supplementary Source*

The google-play-scraper library provides access to live Play Store reviews for any app without requiring API keys or authentication. We targeted com.whatsapp across both English (US) and English (India) locales.

The collector fetches the newest 5,000 reviews per language/country combination and applies strict date filtering to keep only records within our Nov 2020 – Jul 2021 analysis window. Star ratings are captured as an engagement proxy. Note: Google's Play Store API enforces recency limits, so historical coverage is best-effort; the Kaggle dataset is the primary Play Store source.

■ Collector Script: `google_play_reviews_collector.py`

```
"""
google_play_reviews_collector.py - Google Play Store Review Collector
Data Vortex A'26 | Team: Event Horizon

Scrapes user reviews for com.whatsapp from Google Play (free, no auth).
Filters by date range and languages/countries defined in config.

Required env vars: None
Dependency: google-play-scraper
"""

import sys, os
sys.path.insert(0, os.path.join(os.path.dirname(__file__), ".."))

import time
import pandas as pd
from datetime import datetime, timezone
from config import (
    GOOGLE_PLAY_APP_ID, GOOGLE_PLAY_LANGS, GOOGLE_PLAY_COUNTRIES,
    DATE_START, DATE_END, DATA_RAW_DIR, COLL_LOG_PATH
)

try:
    from google_play_scraper import reviews, Sort
    GP_AVAILABLE = True
except ImportError:
    GP_AVAILABLE = False
    print("[WARN] google-play-scraper not installed. Run: pip install google-play-scraper")

DATE_START_DT = datetime.strptime(DATE_START, "%Y-%m-%d").replace(tzinfo=timezone.utc)
DATE_END_DT = datetime.strptime(DATE_END, "%Y-%m-%d").replace(tzinfo=timezone.utc)

def log_collection(source, query, count, status, run_ts=None):
    run_ts = run_ts or datetime.now(timezone.utc).isoformat()
    row = pd.DataFrame([
        {
            "timestamp": run_ts,
            "source": source,
            "query": query,
            "count": count,
            "status": status,
        }
    ])
    if os.path.exists(COLL_LOG_PATH):
        row.to_csv(COLL_LOG_PATH, mode="a", header=False, index=False)
    else:
        os.makedirs(os.path.dirname(COLL_LOG_PATH), exist_ok=True)
        row.to_csv(COLL_LOG_PATH, index=False)

def collect_google_play(max_per_combo: int = 5000) -> pd.DataFrame:
    if not GP_AVAILABLE:
        log_collection("google_play", GOOGLE_PLAY_APP_ID, 0, "SKIP_NO_LIB")
        return pd.DataFrame()

    os.makedirs(DATA_RAW_DIR, exist_ok=True)
```

```

run_ts = datetime.now(timezone.utc).isoformat()
all_rows = []

for lang in GOOGLE_PLAY_LANGS:
    for country in GOOGLE_PLAY_COUNTRIES:
        print(f" [GP] lang={lang} country={country} ...")
        try:
            result, _ = reviews(
                GOOGLE_PLAY_APP_ID,
                lang=lang,
                country=country,
                sort=Sort.NEWEST,
                count=max_per_combo,
            )
            kept = 0
            for r in result:
                at = r.get("at")
                if at is None:
                    continue
                if hasattr(at, "tzinfo") and at.tzinfo is None:
                    at = at.replace(tzinfo=timezone.utc)
                elif not hasattr(at, "tzinfo"):
                    continue
                if not (DATE_START_DT <= at <= DATE_END_DT):
                    continue
                all_rows.append({
                    "post_id": r.get("reviewId", ""),
                    "source": "google_play",
                    "source_type": "bonus_live_scraped",
                    "source_name": "google_play",
                    "text": r.get("content", ""),
                    "timestamp": at.isoformat(),
                    "engagement_metric": float(r.get("thumbsUpCount", 0)),
                    "star_rating": r.get("score", 0),
                    "country": country,
                    "lang": lang,
                    "query_used": f"{lang}_{country}",
                })
                kept += 1
            print(f" -> {kept} reviews in date range")
            time.sleep(1.0)
        except Exception as e:
            print(f" [GP] ERROR lang={lang} country={country}: {e}")
            log_collection("google_play", f"{lang}_{country}", 0, f"ERROR:{e}", run_ts)

df = pd.DataFrame(all_rows)
if not df.empty:
    df.drop_duplicates(subset=["post_id"], inplace=True)
    ts_label = datetime.now(timezone.utc).strftime("%Y%m%dT%H%M%S")
    out_path = os.path.join(DATA_RAW_DIR, f"google_play_{ts_label}.csv")
    df.to_csv(out_path, index=False)
    print(f" [GP] Saved {len(df):,} reviews -> {out_path}")
    log_collection("google_play", GOOGLE_PLAY_APP_ID, len(df), "OK", run_ts)
else:
    print(" [GP] No reviews in date range.")
    log_collection("google_play", GOOGLE_PLAY_APP_ID, 0, "EMPTY", run_ts)

return df

if __name__ == "__main__":
    print("=== Google Play Collector - WhatsApp Reviews Jan-May 2021 ===")
    df = collect_google_play()
    print(f"Total rows collected: {len(df):,}")

```

5. Reddit (PRAW)

◆ [Live API](#) • [OAuth Required](#) • [Supplementary Source](#)

Reddit hosts some of the most substantive public discourse around technology and privacy. Subreddits like r/whatsapp, r/privacy, r/technology, and r/news saw heavy activity during the January 2021 policy announcement.

We used PRAW (Python Reddit API Wrapper) with a registered script-type application (client_id + client_secret stored as environment variables). The collector searches five keyword queries across four subreddits, keeping only posts within our date window. Post title and self-text are concatenated into the text field; upvote score serves as the engagement metric. A 0.6-second politeness delay is applied between every API call.

■ Collector Script: `reddit_collector.py`

```
"""
reddit_collector.py - Reddit PRAW Collector
Data Vortex A'26 | Team: Event Horizon

Fetches submissions and comments from relevant subreddits mentioning
WhatsApp privacy keywords (Jan-May 2021).

Required env vars:
    REDDIT_CLIENT_ID      - from reddit.com/prefs/apps (script type)
    REDDIT_CLIENT_SECRET  - from reddit.com/prefs/apps
"""

import sys, os
sys.path.insert(0, os.path.join(os.path.dirname(__file__), ".."))

import time
import pandas as pd
from datetime import datetime, timezone
from config import (
    REDDIT_CLIENT_ID, REDDIT_CLIENT_SECRET, REDDIT_USER_AGENT,
    KEYWORDS, GEOGRAPHY, DATE_START, DATE_END,
    DATA_RAW_DIR, COLL_LOG_PATH
)

DATE_START_TS = datetime.strptime(DATE_START,
    "%Y-%m-%d").replace(tzinfo=timezone.utc).timestamp()
DATE_END_TS   = datetime.strptime(DATE_END,
    "%Y-%m-%d").replace(tzinfo=timezone.utc).timestamp()

def log_collection(source, query, count, status, run_ts=None):
    run_ts = run_ts or datetime.now(timezone.utc).isoformat()
    row = pd.DataFrame([{
        "timestamp": run_ts,
        "source": source,
        "query": query,
        "count": count,
        "status": status,
    }])
    if os.path.exists(COLL_LOG_PATH):
        row.to_csv(COLL_LOG_PATH, mode="a", header=False, index=False)
    else:
        os.makedirs(os.path.dirname(COLL_LOG_PATH), exist_ok=True)
        row.to_csv(COLL_LOG_PATH, index=False)

def get_reddit_instance():
    if not REDDIT_CLIENT_ID or not REDDIT_CLIENT_SECRET:
        raise EnvironmentError(
            "Reddit credentials missing. Set REDDIT_CLIENT_ID and "
            "REDDIT_CLIENT_SECRET environment variables.\n"
            "Register a free app at: https://www.reddit.com/prefs/apps"
        )
    )
```

```

import praw
return praw.Reddit(
    client_id=REDDIT_CLIENT_ID,
    client_secret=REDDIT_CLIENT_SECRET,
    user_agent=REDDIT_USER_AGENT,
    ratelimit_seconds=300,
)

def collect_reddit(limit_per_query: int = 500) -> pd.DataFrame:
    os.makedirs(DATA_RAW_DIR, exist_ok=True)
    run_ts = datetime.now(timezone.utc).isoformat()
    all_rows = []

    try:
        reddit = get_reddit_instance()
    except EnvironmentError as e:
        print(f" [Reddit] SKIP: {e}")
        log_collection("reddit", "all", 0, "SKIP_NO_CREDS", run_ts)
        return pd.DataFrame()
    except Exception as e:
        print(f" [Reddit] SKIP: {e}")
        log_collection("reddit", "all", 0, f"ERROR:{e}", run_ts)
        return pd.DataFrame()

    subreddits = GEOGRAPHY["reddit_subreddits"]
    search_queries = [
        "WhatsApp privacy policy",
        "WhatsApp new terms",
        "WhatsApp privacy",
        "delete WhatsApp",
        "WhatsApp ban",
    ]

    for sub_name in subreddits:
        for query in search_queries:
            print(f" [Reddit] r/{sub_name} | '{query}'")
            try:
                subreddit = reddit.subreddit(sub_name)
                results = subreddit.search(
                    query,
                    sort="new",
                    time_filter="all",
                    limit=limit_per_query,
                )
                kept = 0
                for post in results:
                    ts = post.created_utc
                    if not (DATE_START_TS <= ts <= DATE_END_TS):
                        continue
                    created_iso = datetime.fromtimestamp(ts, tz=timezone.utc).isoformat()
                    text = f"{post.title} {post.selftext}".strip()
                    if not text:
                        continue
                    all_rows.append({
                        "post_id": post.id,
                        "source": "reddit",
                        "text": text,
                        "timestamp": created_iso,
                        "engagement_metric": float(post.score),
                        "num_comments": post.num_comments,
                        "subreddit": sub_name,
                        "url": f"https://reddit.com{post.permalink}",
                        "country": "",
                        "lang": "en",
                        "query_used": query,
                    })
                    kept += 1
                print(f" -> {kept} posts in date range")
                time.sleep(0.6)
            except Exception as e:
                print(f" [Reddit] ERROR r/{sub_name} '{query}': {e}")
                log_collection("reddit", f"r/{sub_name}:{query}", 0, f"ERROR:{e}", run_ts)

    df = pd.DataFrame(all_rows)

```

```

if not df.empty:
    df.drop_duplicates(subset=["post_id"], inplace=True)
    ts_label = datetime.now(timezone.utc).strftime("%Y%m%dT%H%M%S")
    out_path = os.path.join(DATA_RAW_DIR, f"reddit_{ts_label}.csv")
    df.to_csv(out_path, index=False)
    print(f" [Reddit] Saved {len(df):,} posts -> {out_path}")
    log_collection("reddit", "multi-query", len(df), "OK", run_ts)
else:
    print(" [Reddit] No posts collected.")
    log_collection("reddit", "multi-query", 0, "EMPTY", run_ts)

return df

if __name__ == "__main__":
    print("=== Reddit Collector - WhatsApp Privacy Jan-May 2021 ===")
    df = collect_reddit()
    print(f"Total rows collected: {len(df):,}")

```